

An Efficient Hardware Realization of Diamond Search Algorithm for Motion Estimation Task in Video Compression Applications

Ganapathi Hegde and Bichu Vijay

Department of Electronics and Communications Engineering

Amrita school of Engineering

Amrita Vishwa Vidyapeetham

Bengaluru, India

Email: ganapathi_hegde@blr.amrita.edu and vijaybichu.in@gmail.com

Abstract—This work presents an efficient hardware realization of diamond search motion estimation algorithm using adder compressor units. The main advantage of this kind of hardware is its large quantity of processing modules (PM) which are employed to determine the sum of absolute difference distortion metric. Processing units are composed of large quantity of adders to calculate the summation of absolute difference (SAD). Incorporated 3:2 and 5:2 adder compressor units (ACU) are used in the processing units to achieve high computational speed and make the architecture compatible for high definition television videos. The adder compressor units are capable of parallel addition of eight addend data. The complete hardware is realized in Verilog HDL, functionally verified, and implemented on FPGA. Also, the design is synthesized using Synopsys RC design compiler of 90nm technology standard cell library as an application specific integrated circuits (ASIC) implementation. Simulation and synthesis results show that the implemented architecture for diamond search algorithm achieve best results when performance evaluation is done.

Keywords—Block matching, Motion estimation, Diamond Search, Adder compressor.

I. INTRODUCTION

Presence of high extra information in an image and video data needs to be discarded. The inter-pixel redundancy in spatial coordinates needs intra-frame coding. But inter-pixel redundancy in temporal domain requires complex inter-frame coding. Therefore, video compression employs both intra-pixel and inter-frame coding techniques. The main challenge in video compression is efficient reduction of the temporal redundant data. Some of the compression techniques applicable to video are entropy coding, transform coding, vector quantization, predictive coding, and motion estimation/compensation. Among all these techniques, motion estimation (ME) is most specific to video coding. In getting the video coding standards, ME algorithms play major role in achieving high level of compression by removing temporal redundancy contained by video data. ME task offers very high computational hardness and it is a fundamental function in video compression. There are several available algorithms for ME task and significant among them is full search matching (FS) algorithm.

It discovers the best match for the whole search window by calculating the SAD at each pixel spot [1]–[4]. For a search window of magnitude $\pm w$ pixels, the number of search locations that need to search are $(2w+1)^2$. As the size of the search window increases, searching the locations to procure a best match with a least SAD value results in a substantial computational complexity. It utilizes maximum computing power of the encoder [5], [6]. To cut-down these many complexity involved in computation, different improved search algorithms can be usable like three step search (TSS), new three step search, four step search, block based gradient descent search, and diamond search (DS) etc. [7], [8]. TSS dilutes the number of reckons by using a coarse to fine search scheme. Other algorithms remarked above have less number of estimations when compared with TSS by using middle tendency biased motion vector (MV) spread attributes [9]. In real time video frame succession, the motion of macro-blocks is as stationary or semi-still. If blocks are stationary, then related MVs are located at the search center. Further, if the blocks are quasi-stationary, then comparable MVs are enclosed in a window of size 2 pixel distance around the center. The real time video processing requires tremendous computational capabilities that can only be achieved cost effectively by using specific VLSI architecture [10]. Utility of ME algorithms strongly depends on the practicability and activeness of its VLSI implementation. Implementation of an efficient SAD calculation block is necessary to improve the performance of ME architecture. Since the PUs consists of large quantity of additions to compute the SAD metric, efficient adders are used to cut-down the computational complexity and further increase the performance of the PUs. A group of 3:2 and 5:2 ACUs, which execute the concurrent addition of eight addends, are used in the new solutions for PM structures. Results show that the use of 3:2 and 5:2 compressors significantly improves the performance of the architecture of DS algorithm for ME task [11]. Moreover, improvement in performance is in terms of processing rate and hardware utilization, when compared to the other architectures. After analyzing, the attributes of the MVs, the search pattern of large diamond search pattern (LDSP)

comprises of nine search points with pixel distances of 2 and small diamond search pattern (SDSP) consists of five search points with a pixel distance of 1 based on the origin [12]. We project low cost optimized high level functioning structure for DS algorithm used for high definition video diligence. Advantage of the proposed hardware realization is the huge quantity of PMs that are used to calculate the SAD metric. In this work, we have considered a DS algorithm as a substitute for fast block ME algorithm. Rest of the paper is presented as follows. Section-II describes about the block matching ME algorithms. Section-III introduces about the DS algorithm used for ME task here. Section-IV then describes the proposed hardware realization of DS algorithm required for MV computation. Execution outcomes and operating capability evaluation are discussed in section-V. The paper is concluded based on the results in section-VI.

II. BLOCK MATCHING MOTION ESTIMATION TECHNIQUES

Block matching algorithms (BMA) are required for ME task matching technique carry out on block by block basis. They are used in many video coding criteria like MPEG-1, MPEG-3, H.261, and H.263. In BMA image frames are fragmented into several non-shingle macro-blocks and best matches is searched within the search range by using all possible position for each macro block. In real time video, motion can be a complex interconnection of translational and rotational movement of pixels. Such a type of motion is hard to be approximated and require large complexity in operation. Translational motion in a video can be clearly estimated and is used effectively for motions expiate coding. In block matching technique of the ME, frame is divided in to $n \times n$ macro blocks. In general for a best macro block match search area restricted up to p pixel on every four edges of the conforming candidate block of former frame. This p is referred as the check parameter. For large motions, the macro block instruct a greater value of p , and hence the search parameter of the ME process require large number of computation. Block based motion estimation technique followed for successive frames are shown in Fig 1. Equalizing of each macro-block with the other is stand on the outcome of a matching purpose. The macro- block that effect in the least SAD is the one which relates closest to the existing block. If the value of n is increases, then the number of blocks in each frame which required for operation decreases. Therefore, computational complexity corresponding to this also decreases.

III. DS ALGORITHM

DS algorithm employ two diamond structured search schemes having enormous dimensions. They are depending on the arrangement that MVs are middle biased. Algorithm employs two search schemes (Fig. 2). An LDSP consists of nine points and an SDSP of five points respectively. DS algorithm starts with an LDSP strategy. LDSP is input to reiterate until the corresponding reference value in which the least SAD point happens at the middle of the diamond. The hunting strategy is then changed to SDSP. The point having

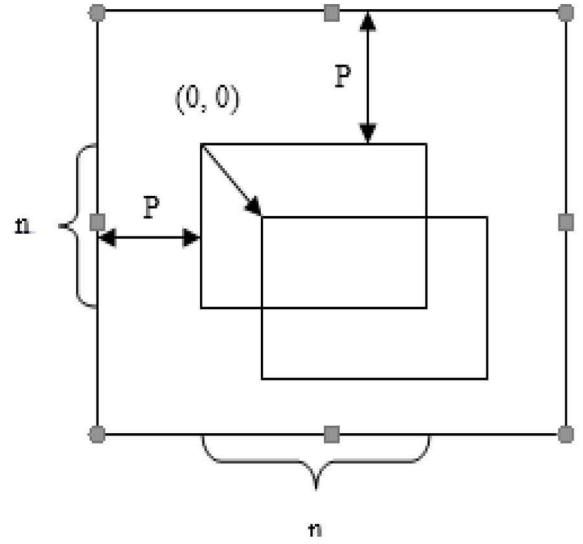


Fig. 1. Block matching process in search window

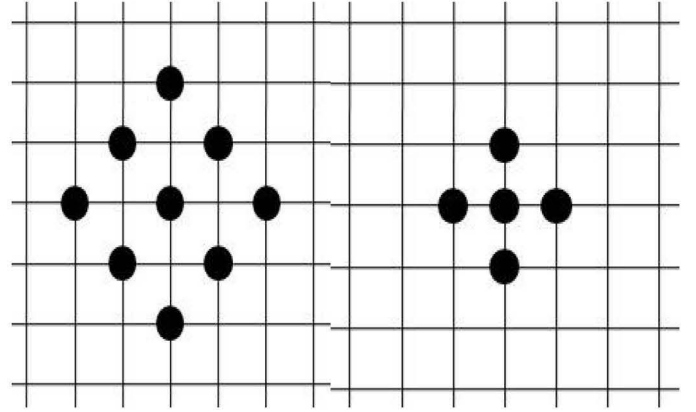


Fig. 2. LDSP and SDSP schemes adapted in DS algorithm

minimum cost metric among the five verifying points in the SDSP gives the MV of the accurate matching block. LDSP and SDSP schemes adapted in DS algorithm are shown in Fig. 2. Cost function which is used in the algorithm is SAD and is given in equation (1).

$$SAD = \sum_{i=0}^{N-1} \sum_{j=0}^{N-1} |C_{ij} - R_{ij}| \quad (1)$$

In SAD corresponding pixels from each block are marked and their deviations are summed-up, as given in (1). In this equation N is the dimension of the macro block, C_{ij} and R_{ij} are the pixel intensities that are compared between the present macro block and allusion macro block, pertinently. This SAD function is for two blocks C and R of size $N \times N$. C_{ij} is the value of pixel in the i th row and j th column of the block C . The lower the SAD, the better is the match and so the candidate block with least SAD should be selected. Peak signal to noise ratio (PSNR) is an extent metric that characterizes the motion

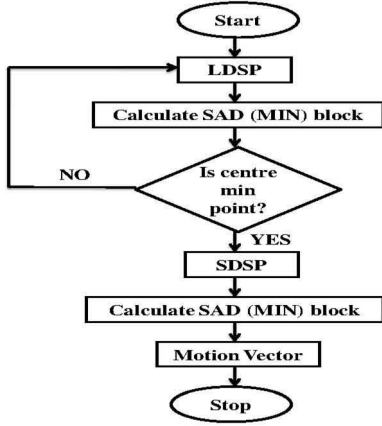


Fig. 3. Flow-chart of DS algorithm

reparation image that is reconstructed by utilizing MVs. These MVs are among the macro blocks which are taken from the quoted frame. It is stated in equation (2).

$$PSNR = 10 \log_{10} \left[\frac{255^2}{MSE} \right] \quad (2)$$

The summarized DS algorithm is represented as a step by step procedure is provided below. Step(i): SAD is computed for each nine search points in the LDSP. If the least SAD value exists at the middle position of LDSP, then go to step(iii) otherwise go to step(ii). Step(ii): Least SAD point is at the middle, create new LDSP, and repeat same procedure followed in step-i. Step(iii): Change search scheme from LDSP to SDSP. Least SAD points exists during this step are final solution for MV. Flowchart showing the above steps in an algorithmic fashion for a DS algorithm is shown in Fig. 3. Here, computation of the LDSP is the first step. If the least SAD value is not at the center point, then LDSP need to be calculated again. If the center minimum point is same as least SAD value, then it is nothing but SDSP. From the available SDSP, least SAD is computed and the outcome of this is nothing but MV (Fig. 3). The same procedure is repeated for entire frame and there will be one MV for each block.

IV. ARCHITECTURE FOR DS ALGORITHM

The specialty of this architecture is it consists of large number of PUs which are deployed to calculate the SAD metric. So, the execution of an efficient SAD computation unit is necessary to improve the performance of ME structure. PMs are made up of huge numeral of adders to assess the SAD metric and their responsibility is to decrease the ascertaining complication of the architecture and in turn improve the performance of the PMs. A mix of 3:2 and 5:2 ACUs, which execute the parallel uniting of eight addend data, are used as a new solution for PM architecture [10]. The new PUs having combination of 3:2 and 5:2 ACUs which are input to the architecture for DS algorithm is shown in the Fig. 4. This architecture can be used for HDTV which offers videos with a low hardware cost. The result analysis shows that the usage of

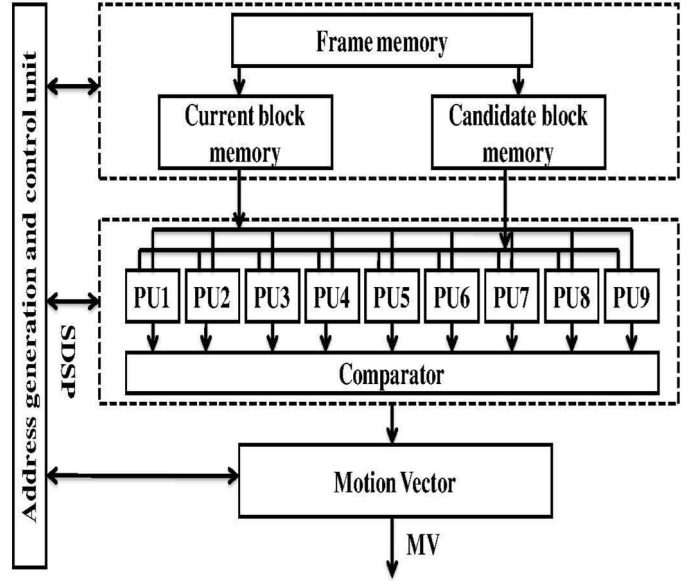


Fig. 4. Architecture of the complete DS algorithm

3:2 and 5:2 ACUs can significantly improve the performance of the architecture. Distortion criteria used to obtain the MV for this architecture is minimum SAD. The processing unit consists of SAD computation module and comparison module. SAD module computes the sum of absolute difference value from the contemporary block and the reference block by comparing the intensities and then takes absolute difference of the relevant pixel values taken from both the blocks. The SAD computed for each block is collected in a register and it is associated with the existing least SAD stored in another register. MV unit will save the values of the MVs for each of the macro block present in the current frame. The architecture consists of nine PMs for each of the candidate blocks. These candidate blocks of the LDSP are assessed in congruent and the outcomes are finally sent to the comparator. Each candidate block accepts an index to classify the stance of this block in the LDSP. Comparator identifies the least SAD and deposes the index of the selected candidate block to the control unit. The control unit analyses the block index and determines the next step of the block matching process. If the selected block contains index zero that then SAD is identified in the center of the LDSP. Then SDSP flag is set and the SDSP pattern is applied. When the selected block is in the first step and also index not at the center, then control unit starts the second step of the algorithm with a inspect pattern forming new LDSP with prior lowest point as the center. The second step can be reiterated n times, till the minimum SAD is found in the center of the LDSP. Then again SDSP flag is set and it is applied. In the SDSP value estimation, four extra candidate block intensities are appraised. The least SAD is extracted and the control unit finally sends the relevant MV related to this block.

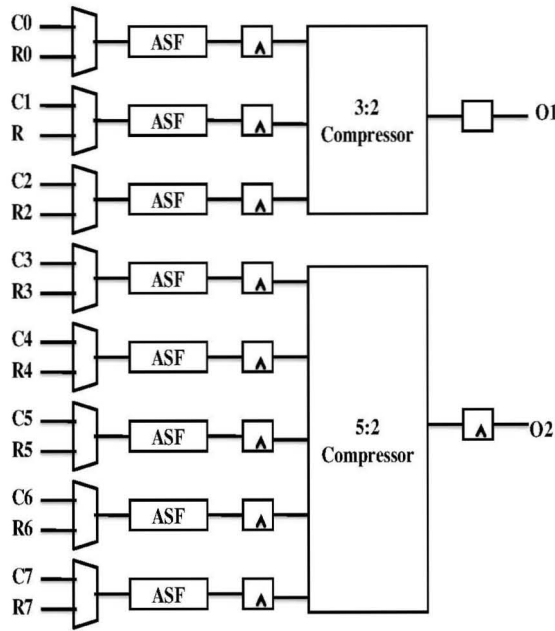


Fig. 5. Block schematic of the PU architecture

A. Processing unit

In this architecture PU is employed to compute addition which consists of 3:2 and 5:2 ACUs [11]. Large number of additions are effectively executed by adder-compressors. For a conventional ME architecture, computation demands processing of eight operands simultaneously. This difference here is due to the usage of adders which are present in PUs. The architecture considered here uses optimal ACUs to perform additions. Pictorial representation of the PM shape is presented in Fig. 5. Two separate ACUs namely 3:2 and 5:2 are employed to execute the sum of eight operands which are available for the output of one level of column registers. The sum value generated by these 3:2 and 5:2 ACUs would represent the module of the subtractions that are performed at each pair of the input operands. Since eight parallel additions are carried out by the 3:2 and 5:2 ACUs, it is able to remove two columns of adders, when compared with additions that are performed using an adder tree. Thus, adder compressor technique saves one clock cycle when compared with the PUs of 4:2 architecture and equals to clock cycle of PU of 8:2 architectures [8]. The SAD module calculates the summation of absolute differences from the current block and the reference block after suitable comparison and then considering absolute difference of the relevant pixel luminance in both the blocks. Finally it accumulates 64 absolute differences for the (8x8) block. Block schematic of inner framework of the 3:2 ACU is shown in Fig. 6. 3:2 ACU have three inputs and two outputs (nothing but a full adder).

5:2 ACU have seven inputs and two outputs. The first three absolute differences are given to 3:2 ACU. Five absolute difference output and two outputs are taken from the 3:2 adder compressor and is given as an input to 5:2 ACU. The final

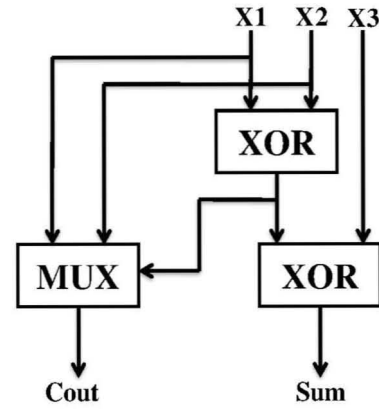


Fig. 6. Block schematic of 3:2 ACU

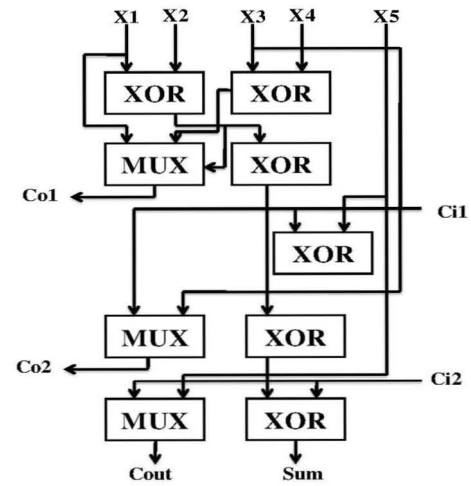


Fig. 7. Structure of 5:2 ACU

output (sum) from the adder comes out from the 5:2 adder compressors. The 5:2 ACU utilize six XOR gates and 3:2 ACU utilize two XOR gates separately. So, a total of eight XOR gates delay need to be added cumulatively to produce the final output of each PM. Block schematic of the inner configuration of the 5:2 ACU is depicted in Fig. 7.

All the pixel values considered here are of length 8-bits. So the maximum possible SAD value is equal to $256 \times 64 (=16384)$ which demands at least 14-bit register to store this data. If the address of the reference block descends out of the boundary, then the SAD register is filled-up with an utmost feasible register value (i.e. 16384). The address of the block is infirm if any of its pixels are located farther the available limit of column address. It computes the difference among two image blocks on pixel by pixel basis and then computes the absolute value. 64 absolute values are added in a adder to get complete sum of absolute difference. The diagrammatic representation of step by step procedure to compute least SAD and the block schematic corresponding to its hardware are elucidated in Fig. 8 and Fig. 9 respectively.

When compared with the 4:2 hardware based adder com-

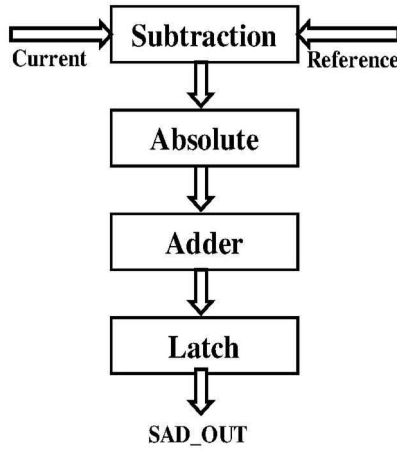


Fig. 8. Step by step procedure to compute SAD

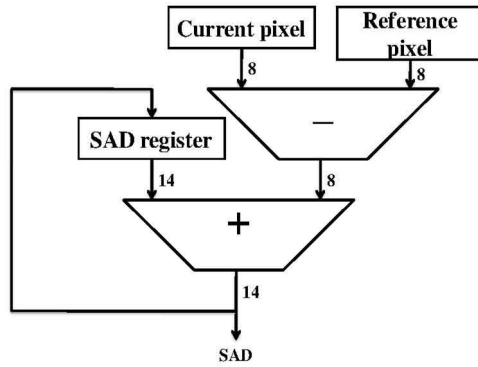


Fig. 9. Block schematic of the SAD unit

processor one XOR gate count can be saved. So, hardware cost of the 3:2 and 5:2 ACU saves XOR gate delay. When compared with the PU of 8:2 hardware it saves four XOR gates that can be summarily eliminated in the design. Hence it proves to be reduction in the area of utilization of the architecture. Block schematic of the comparator which is used in the DSMA architecture shown in the Fig. 10. The 14-bit valued nine SAD metric is compared and the least SAD value is found out from it. Sign bit taken from select output will determine the selection of the least SAD value. This minimum value is stored in a 14-bit register (Fig. 9). According to address of the macro block, the control unit will assign next level address to evaluate the corresponding frame (video data).

B. Address generator

Address generation block is the major part of the control unit that generates the addresses to refer the pixel intensity values for further processing, which is already stored in the frame memory. Representational graphic of the address generation unit is indicated in Fig. 11.

Base address generator and reference address generator are the two units which are used to store the base and reference address respectively. They are used to generate addresses of the current macro block for block matching process and the

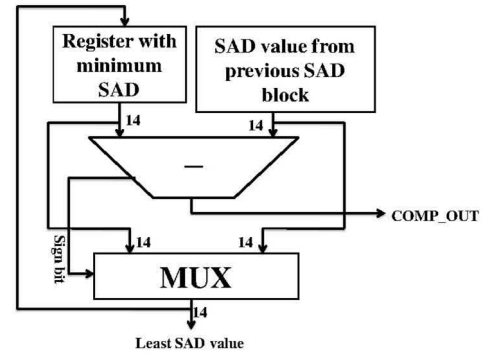


Fig. 10. Block schematic of the comparator unit

addresses of the nodes in the form of mesh based pattern for ME task. This block will provide the 16-bit address for the prevailing block and the reference block which in turn goes to the block address generator unit. Block address generator unit is demand to produce counterpoise addresses, with reference to the given block address and in sequence to scan particular pixels of the given 8x8 block in raster scan arrangement (Fig. 10). Further, it considers the address of an 8x8 block as an input and produces the address to the block of pixels. Address of the input is nothing but the location of the top left edge pixel of the given 8x8 matrix block. Each and every pixel of the arranged 8x8 block is taken in to account using a counter.

C. Memory generation

PU receives data from the stored memory of the candidate block and the current block (Fig. 4-Fig. 7). Each PU receives one word each from the candidate block and the current blocks respectively and then computes the divergence among the eight samples which are present in each of these words. First, eight stimulants from the current block is subtracted from the candidate block. The absolute values are taken from the divergence between these samples and they are summed-up. Final result is obtained by the collection of these divergence (for all the stimulants of the line) which are stored in the output register. Current block consists of memory of size 64 bytes to store the macro block in the form of 8x8 matrix. Candidate block requires a memory of size 256 bytes to store 16x16 candidate blocks. Memory unit receives input data in the form of control signal from control unit and pixel location information from address unit for further processing.

V. EXPERIMENTAL RESULTS

Soft simulation is performed to evaluate the approximate count of search dots per image piece on an average and mean SAD value per pixel. Experimental outcomes are estimated relatively in table-I. Complexity involved in MV computation is obtained depending on the luminance of the pixel in a block of 16x16 for search region and the range of the examining window considered is 7 (=w) pixels. Simulation is carried out on all the three video sequences. These algorithms are implemented in MATLAB and PSNR is taken.

TABLE I
AVERAGE PSNR (DB) PER PIXEL

Video scene	FS	TSS	DS
Caltrain	27.396	27.296	27.219
Misa	37.125	37.000	37.064
Gess	35.432	35.192	35.423

TABLE II
AVERAGE NUMBER OF COMPUTATIONS FOR DIFFERENT ALGORITHMS

Video scene	FS	TSS	DS
Caltrain	207.410	23.275	18.416
Misa	204.282	23.287	13.941
Gess	204.283	23.212	12.707

TABLE III
SYNTHESIS RESULTS FOR IMPLEMENTATION ON FPGA

Parameter	Porto [11]	Proposed
Number of slices	2007	1597
Number of slices FF	2086	1834
Number of 4 input LUT	3610	3423
Minimum period (ns)	4.688	4.582
Maximum frequency (MHz)	213.3	218.21

TABLE IV
AREA REPORT GENERATED BY HARDWARE

Area parameter (Square micrometer)	Area utilization
Combinational area	16607.23
Non-combinational area	12366.95
Net interconnect area	2201.28
Total cell area	28974.18
Total area	31175.46

Results prove that performance exhibited by the DS algorithm is almost near to that of FS algorithm. However, it requires on an average less number of searches per macro block when compared to other ME algorithms (Table-II). Hence, number of computation required for DS algorithm is also less when compared to other ME algorithms. The designed architecture is realized in Verilog HDL and implemented on Xilinx FPGA Virtex-4 (Device: xc4vlx15). Table-III reports the synthesized results generated for the architecture implemented on FPGA. Hardware utilization of the recommended method is 3423 look up tables (LUT) . This hardware consumes 1597 slices. Digital video format considered for all the video scene are quarter common intermediate format (QCIF) standard.

The power and area report generated by Synopsys design compiler for architecture of DS algorithm using 90nm technology are proffered in table-IV and table-V particularly. Total area of utilization is 33175.46(m)² and power dissipation is 137.53W. This application specific integrated circuit proves that it gives good results.

MVs generate by DS algorithm ME architecture for HD video frame employing 5:2 3:2 ACU is shown in the Fig. 12

TABLE V
POWER REPORT OF THE ARCHITECTURE

Power parameter	Power consumption (μW)
Total power	137.53

VI. CONCLUSION

In this work, architecture of DS algorithm for ME task is designed. This architecture uses an alternative for the PU needed for SAD computations. The processing unit of the DSMA architecture is developed using 3:2 and 5:2 adder compressor in order to improve adder computation. The designed DSMA architecture operates at a frequency of 218.21MHz on FPGA. The architecture can be compatible for HD video standards. In terms of device utilization the architecture offers good improvement and power consumption of 137.53W. The DS algorithm offers better performance as compared to full search and other fast algorithm. It uses diminished amount of search points to find out minimum SAD. PSNR of DS algorithm is good when compared to other ME algorithms.

REFERENCES

- [1] Y. Ding and X. L. Yan, "Parallel architecture of motion estimation for video format conversion with center-biased diamond search," in *2009 International Conference on Information Engineering and Computer Science*, Dec 2009, pp. 1–4.
- [2] G. Sanchez, F. Sampaio, M. Porto, S. Bampi, and L. Agostini, "Dmpds: a fast motion estimation algorithm targeting high resolution videos and its fpga implementation," *International Journal of Reconfigurable Computing*, vol. 2012, p. 5, 2012.
- [3] O. Ndili and T. Ogunfunmi, "Algorithm and architecture co-design of hardware-oriented, modified diamond search for fast motion estimation in h. 264/avc," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 21, no. 9, pp. 1214–1227, 2011.
- [4] S. R. Devi, P. Rangarajan, and J. R. P. Perinbam, "Vlsi implementation of high performance optimized architecture for video coding standards," *Acta Polytechnica Hungarica*, vol. 10, no. 6, pp. 237–249, 2013.
- [5] M. Porto, L. Agostini, S. Bampi, and A. Susin, "A high throughput and low cost diamond search architecture for hdtv motion estimation," in *Multimedia and Expo, 2008 IEEE International Conference on*. IEEE, 2008, pp. 1033–1036.
- [6] S. M. Hashimaa, I. I. Mahmoud, and A. A. Elazm, "Hardware implementation of diamond search algorithm for motion estimation and object tracking," in *Proc. of the 7th Conference on Nuclear and Particle Physics*, 2009.
- [7] S. Zhu and K.-K. Ma, "A new diamond search algorithm for fast block matching motion estimation," in *Information, Communications and Signal Processing, 1997. ICICS., Proceedings of 1997 International Conference on*, vol. 1. IEEE, 1997, pp. 292–296.
- [8] Z. Liu, Y. Huang, Y. Song, S. Goto, and T. Ikenaga, "Hardware-efficient propagate partial sad architecture for variable block size motion estimation in h. 264/avc," in *Proceedings of the 17th ACM Great Lakes symposium on VLSI*. ACM, 2007, pp. 160–163.
- [9] R. Menon and D. Radhakrishnan, "High performance 5: 2 compressor architectures," *IEE Proceedings-Circuits, Devices and Systems*, vol. 153, no. 5, pp. 447–452, 2006.
- [10] H. G. Musmann, P. Pirsch, and H.-J. Grallert, "Advances in picture coding," *Proceedings of the IEEE*, vol. 73, no. 4, pp. 523–548, 1985.
- [11] M. Porto, A. Silva, S. Almeida, E. Costa, and S. Bampi, "Motion estimation architecture using efficient adder-compressors for hdtv video coding," *Journal of Integrated Circuits and Systems*, vol. 5, no. 1, pp. 78–88, 2010.
- [12] P. Pirsch, N. Demassieux, and W. Gehrke, "Vlsi architectures for video compression-a survey," *Proceedings of the IEEE*, vol. 83, no. 2, pp. 220–246, 1995.